SITUATIONS D'APPRENTISSAGE ET D'ÉVALUATION

SAé 1

Partie 4 : Automatisation (Développement Logiciel)

4. Automatisation de la barrière de passage à niveau 3
4.1 Conception du diagramme d'états 3
4.1.1 Analyse des besoins et contraintes 3
4.1.2 Première approche : La sécurité avant tout (Diagramme à 8 états) 4
4.1.3 Optimisation pour le code : La factorisation (Diagramme à 5 états) 5
4.1.4 Le diagramme final (Respect du Cahier des Charges) 6
4.1.5 Bilan et perspectives 7
4.2 Passage du papier au code (Implémentation) 8
4.2.1 Rendre le code lisible 8
4.2.2 L'astuce de la "Mémoire de sens" 8
4.2.3 Respecter la demande client (Ouverture anticipée) 9
4.3 Fiabilisation et Validation du système 10
4.3.1 Structuration par fonctions (Modularité) 10
4.3.2 La stratégie de validation (Tests Unitaires) 11
4.3.3 Bilan de la validation 11
Conclusion et synthèse de la partie Automatisation 12

4. Automatisation de la barrière de passage à niveau

4.1 Conception du diagramme d'états

L'objectif est de concevoir l'automatisme qui pilote la barrière en fonction des trois capteurs (dpG, dpC, dpD).

4.1.1 Analyse des besoins et contraintes

L'analyse du cahier des charges impose trois règles techniques majeures que notre système doit impérativement respecter :

- **Logique Négative :** Les capteurs envoient un niveau logique 0 lorsqu'un train est présent et 1 lorsqu'il est absent.
- **Symétrie :** Le fonctionnement doit être strictement identique, que le train arrive de la gauche ou de la droite.
- **Condition d'Ouverture :** La barrière doit s'ouvrir dès que la fin du train quitte le capteur central (dpC).

En plus de ces règles, le système doit être capable de s'adapter aux différents gabarits de trains susceptibles de circuler dans la zone de contrôle :

- **Le train court (ou "petit train") :** Sa longueur est inférieure à l'espacement entre les capteurs. Il existe donc un instant critique où il se situe entre deux détecteurs sans en activer aucun. Le système reçoit alors l'information "1 1 1" (comme si la voie était libre) alors que le train est toujours présent.
- **Le train moyen :** Sa longueur lui permet d'activer deux capteurs simultanément lors de sa progression (par exemple dpG et dpC en même temps).
- **Le train long :** Sa longueur est suffisante pour activer les trois capteurs (dpG, dpC et dpD) simultanément lors de son passage.

Parallèlement à cette analyse des gabarits standards, j'ai mené une réflexion sur des contraintes réelles supplémentaires, notamment le cas des **trains discontinus** (présentant des espaces vides entre les wagons ou des wagons plats). J'ai identifié que ces discontinuités pouvaient créer des failles de sécurité dans le programme : un espace vide passant devant le capteur central pourrait être interprété à tort comme la fin du train, provoquant une ouverture prématurée de la barrière.

C'est pour répondre à ce risque spécifique que j'ai orienté ma conception initiale vers une solution de sécurité renforcée.

4.1.2 Première approche : La sécurité avant tout (Diagramme à 8 états)

Pour pallier le risque lié aux trains discontinus et aux "zones mortes" des petits trains, j'ai conçu un premier diagramme à 8 états (voir Figure 4.1.2).

- **Principe :** Ce modèle sépare totalement la séquence "Train venant de Gauche" de la séquence "Train venant de Droite".
- **Avantage Sécuritaire :** Cette distinction permet de mémoriser la provenance du train et d'imposer une condition de sortie stricte. La barrière ne s'ouvre que lorsque le train a libéré le **capteur de sortie** (dpD pour un train venant de gauche, dpG pour un train venant de droite).
- **Résultat :** Le système ignore les potentiels passages à vide au niveau du capteur central et garantit que la zone est totalement libre avant de lever la barrière.

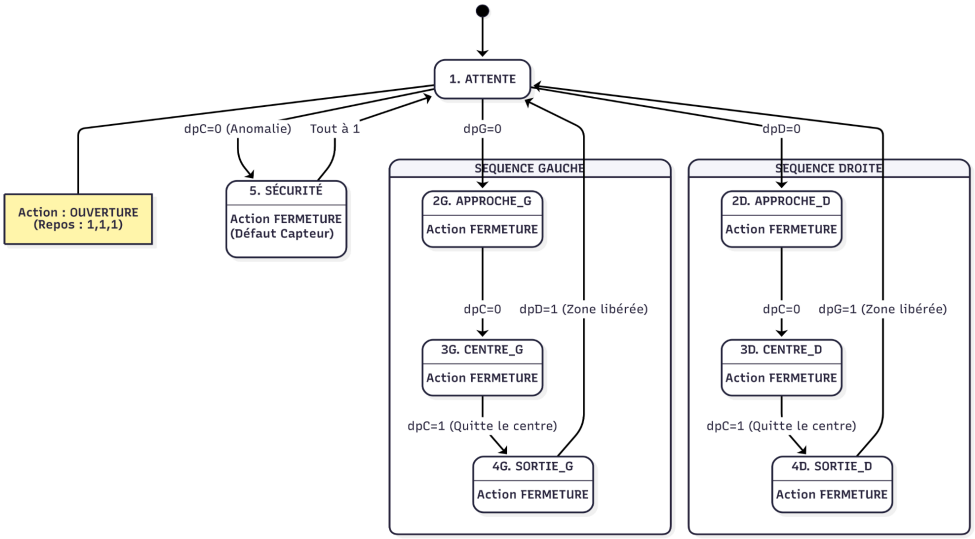

Figure 4.1.2 – Première approche sécurisée à 8 états

4.1.3 Optimisation pour le code : La factorisation (Diagramme à 5 états)

Bien que robuste, le diagramme à 8 états présente une redondance car les actions de fermeture sont identiques dans les deux séquences. Afin d'optimiser le code et de le rendre plus lisible, j'ai factorisé ce modèle pour le réduire à 5 états.

Pour conserver l'information de provenance nécessaire à la gestion du cycle, j'ai introduit une variable mémoire (memoire_sens) :

- **Memoire_sens = 1 :** Le train vient de Gauche.
- **Memoire_sens = 2 :** Le train vient de Droite.

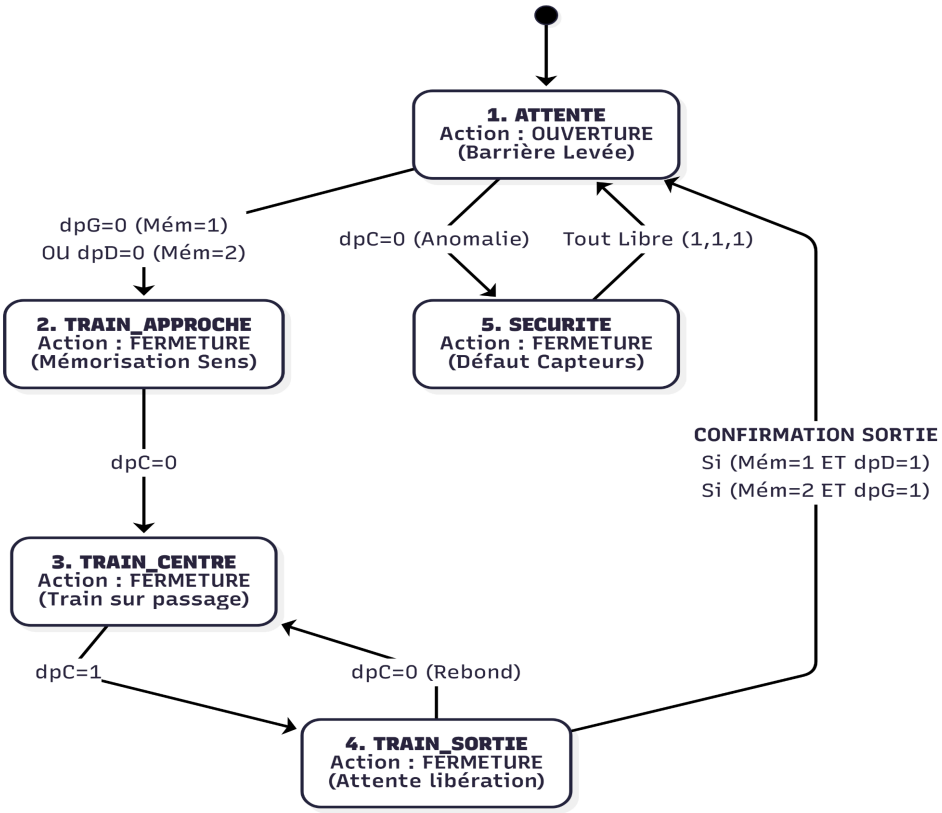

Figure 4.1.3 – Première approche sécurisée à 5 états

4.1.4 Le diagramme final (Respect du Cahier des Charges)

Pour le diagramme final retenu dans ce projet, j'ai adapté mon modèle à 5 états pour qu'il respecte strictement la demande du client (le cahier des charges) : l'ouverture se déclenche dès la libération du capteur central.

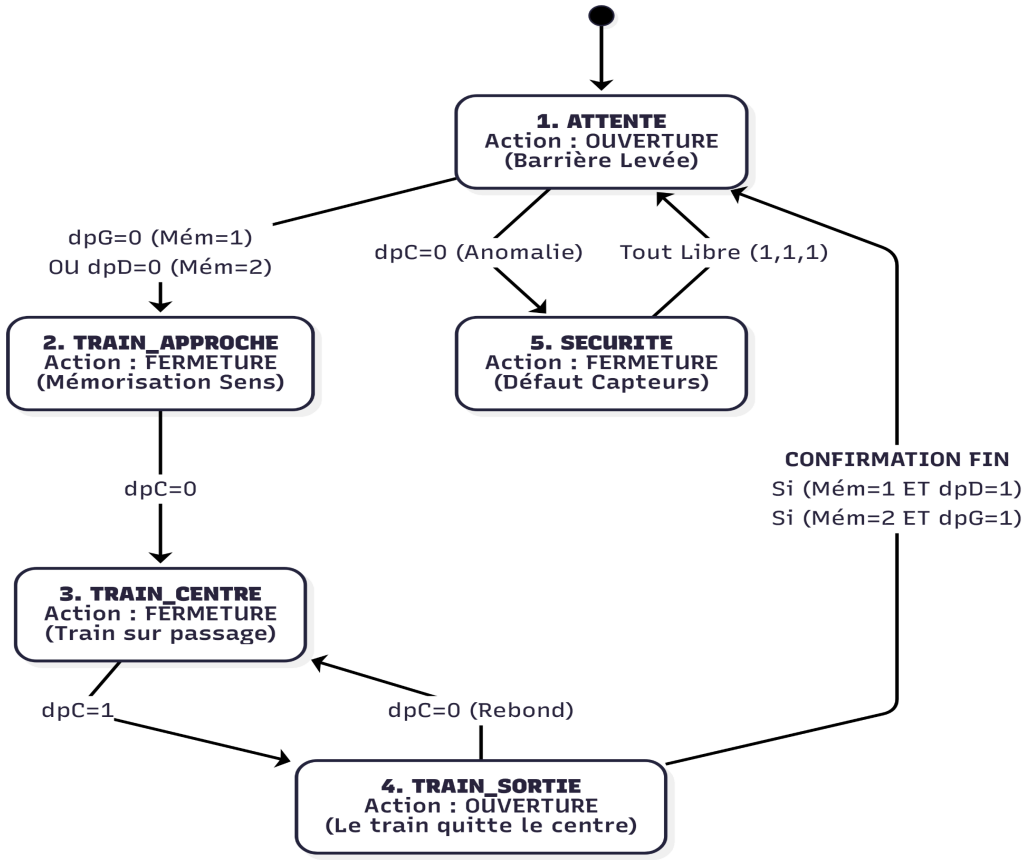

Figure 4.1.4 – Diagramme final optimisé (Conforme au Cahier des Charges)

Dans cette version finale (voir Figure 4.1.4) : L'État 4 (Train Sortie) commande l'**OUVERTURE** de la barrière. Celle-ci se lève dès que le capteur central est libéré, tout en conservant la structure de la machine à états pour gérer les transitions.

4.1.5 Bilan et perspectives

Ce travail de conception m’a permis de mieux comprendre l’importance de la méthodologie de l’ingénieur, notamment dans le respect du cahier des charges.

Dans un premier temps, j’ai orienté ma réflexion vers une solution axée sur une sécurité maximale, en cherchant à anticiper des cas complexes tels que les trains discontinus ou atypiques. Cette approche m’a conduit à concevoir un système plus contraignant que celui demandé initialement, sans répondre en priorité au besoin de base exprimé par le client.

Grâce aux échanges avec mon enseignant, j’ai compris qu’une démarche professionnelle doit avant tout suivre une logique claire : répondre précisément à la demande formulée avant de proposer d’éventuelles améliorations. C’est dans cette optique que le diagramme final à 5 états a été retenu, car il respecte strictement les exigences du cahier des charges, notamment l’ouverture de la barrière dès la libération du capteur central.

Néanmoins, la première solution développée conserve un intérêt technique. Elle peut être envisagée comme une évolution du système, permettant d’améliorer la sécurité dans des situations plus complexes. Ce projet m’a ainsi permis de concilier le respect des contraintes imposées avec une réflexion sur les perspectives d’amélioration du système.

4.2 Passage du papier au code (Implémentation)

Une fois mon diagramme final (Figure 4.1.4) validé, l'étape de programmation est devenue beaucoup plus simple. Comme j'avais déjà défini précisément les états et les flèches sur le papier, écrire le code en C revenait surtout à "traduire" mon schéma : chaque bulle est devenue un cas (`case`) dans mon programme.

4.2.1 Rendre le code lisible

Pour ne pas me perdre avec des chiffres partout (1, 2, 3...), j'ai commencé par donner des noms clairs à mes états en utilisant des #define. Cela rend le code beaucoup plus facile à relire pour un humain.

```
// Définition des états de la machine à état
#define ATTENTE 1        // Barrière ouverte
#define TRAIN_APPROCHE 2 // Train détecté (Gauche ou Droite)
#define TRAIN_CENTRE 3   // Train au milieu
#define TRAIN_SORTIE 4   // Train quitte le milieu (attente confirmation sortie)
#define SECURITE 5       // État de panne/défaut

```

Le cœur de mon programme tourne dans une boucle infinie (while(1)) qui répète toujours les mêmes actions : je lis les capteurs, je calcule l'état suivant, et enfin, j'envoie l'ordre à la carte.

```
while(1)    {

    litEtatDetecteurs(&dpG, &dpC, &dpD);
    etat_precedent = etat;
    switch(etat) {
        // CAS 1 : En attente d'un train
        case ATTENTE:
            memoire_sens = 0; // Reset de la mémoire de sens

            if (dpC == 0) {
                etat = SECURITE; // Anomalie : apparition au centre
            }
            else if (dpG == 0) {
                etat = TRAIN_APPROCHE;
                memoire_sens = 1; // On mémorise : Vient de GAUCHE
            }
            else if (dpD == 0) {
                etat = TRAIN_APPROCHE;
                memoire_sens = 2; // On mémorise : Vient de DROITE
            }
            break;
        // CAS 2 : arrivée du train
        case TRAIN_APPROCHE:
            if (dpC == 0) {
                etat = TRAIN_CENTRE;
            }
        break;
```

4.2.2 L'astuce de la "Mémoire de sens"

Comme je l'ai expliqué dans la partie 4.1.3, je voulais éviter de dupliquer mon code pour la gauche et la droite. C'est là que mon diagramme m'a guidé : j'ai utilisé une simple variable "mémoire" dans le code.

```
32    // Variable mémoire pour savoir d'où vient le train
33    // 0 = inconnu, 1 = vient de GAUCHE, 2 = vient de DROITE
34    int memoire_sens = 0;
```

Concrètement, dès que le train arrive (état ATTENTE), le programme note immédiatement d'où il vient. C'est une partie très pratique du code car elle permet de "factoriser" tout le reste : peu importe si le train vient de gauche ou de droite, les états suivants (CENTRE, SORTIE) sont les mêmes. Le programme se servira de cette mémoire uniquement à la toute fin pour vérifier que le train est bien sorti du bon côté.

4.2.3 Respecter la demande client (Ouverture anticipée)

Enfin, pour piloter la barrière, j'ai choisi de séparer la réflexion (les états) de l'action (le moteur). J'utilise un deuxième bloc `switch` juste pour décider si on ouvre ou si on ferme.

C'est ici que j'applique la règle stricte du cahier des charges : ouvrir dès que le capteur central est libre.

```
// Calcul des sorties en fonction de l'état de la barrière
switch(etat) {
    case ATTENTE:
    case TRAIN_SORTIE:
        commande_barriere = OUVERTURE;
        break;

    // Pour tous les autres cas (Approche, Centre, Sortie, Sécurité) on ferme
    case TRAIN_APPROCHE:
    case TRAIN_CENTRE:
    case SECURITE:
        commande_barriere = FERMETURE;
        break;

    default:
        commande_barriere = FERMETURE; // Sécurité dans le cas ou on est pas en attente dans ce cas on ferme
}
```

J'ai volontairement regroupé l'état `TRAIN_SORTIE` avec l'état `ATTENTE`. Cela permet à la barrière de se lever immédiatement dès que le train quitte le capteur central, comme demandé.

Cependant, si je devais concevoir ce système pour un vrai passage à niveau (type SNCF), je ne ferais pas cela. Pour une sécurité maximale, il faudrait garder la barrière fermée durant la phase `TRAIN_SORTIE` et n'autoriser l'ouverture qu'une fois revenu en état `ATTENTE` (train totalement parti). Cela éviterait tout accident si le train s'arrêtait juste après le capteur central ou s'il s'agissait d'un train avec des wagons espacés. Mais ici, j'ai privilégié le strict respect de la demande initiale.

4.3 Fiabilisation et Validation du système

Une fois la logique de base fonctionnelle, je me suis rendu compte que tout laisser dans le `main` posait un problème : je ne pouvais pas tester mon code sans être branché physiquement à la maquette. Pour répondre aux exigences de qualité du cahier des charges (Exercices 4.4 et 4.5), j'ai restructuré mon programme pour le rendre modulaire et testable.

4.3.1 Structuration par fonctions (Modularité)

Comme demandé dans l'exercice 4.4, j'ai extrait la logique de décision de la boucle principale pour la rendre plus lisible et mieux structurée. Pour cela, j'ai séparé le comportement en deux fonctions, chacune avec un rôle bien défini :

- **calculer_etat_suivant** : c'est le cerveau du système. À partir de l'état courant et des informations fournies par les capteurs, elle décide du prochain état.

- **calculer_sortie** : c'est l'interprète. Elle prend un état abstrait (par exemple *TRAIN_SORTIE*) et le traduit en une commande physique concrète (0 ou 1).

Enfin, les variables de mémoire (**memoire_sens** et **a_touche_sortie**) ont été déplacées directement dans la fonction de transition. Elles y sont déclarées en `static`, ce qui permet de conserver leur valeur d'un appel à l'autre sans les exposer inutilement ailleurs dans le programme.

```
int calculer_etat_suivant(int etat, int dpG, int dpC, int dpD) {
    // Variables mémoires (static pour conserver la valeur entre les appels)
    static int memoire_sens = 0;
    static int a_touche_sortie = 0;
```

Cela permet de protéger ces données : seul le « cerveau » peut les modifier, ce qui limite les risques d'erreurs ou de manipulations accidentelles ailleurs dans le programme.

4.3.2 La stratégie de validation (Tests Unitaires)

L'étape la plus importante a été la mise en place de tests automatiques (Exercice 4.5). L'idée est simple : créer un "simulateur" qui vérifie que mon code réagit correctement à toutes les situations possibles *avant* même d'activer le moteur.

J'ai écrit une fonction `lancer_tests_unitaires` qui s'exécute au démarrage du programme. Elle teste deux types de scénarios :

1. Les scénarios de sécurité (Tests de robustesse) Je vérifie que le système ne fait pas n'importe quoi si les capteurs ont un problème. Par exemple, si tous les capteurs indiquent `0` (détection partout) alors qu'on était au repos, le système doit passer en sécurité et non ouvrir la barrière.

```
// Test Sécurités
if (calculer_etat_suivant(ATTENTE, 1, 0, 1) != SECURITE) succes = 0;
if (calculer_etat_suivant(ATTENTE, 0, 0, 0) != SECURITE) succes = 0;
```

2. Le scénario nominal (Simulation de passage) J'ai simulé le passage complet d'un train virtuel pour vérifier l'enchaînement des états :

- *Repos -> Approche -> Centre -> Sortie.* C'est ici que je valide informatiquement que mon code respecte bien l'ouverture anticipée demandée par le client :

4.3.3 Bilan de la validation

Grâce à cette partie du projet, j'ai pu découvrir les **tests unitaires**, une méthode très efficace pour vérifier que le programme fonctionne correctement dès son lancement. Cette approche m'a particulièrement marqué, au point que je l'utilise désormais aussi dans mes projets personnels.

L'idée est de simuler le comportement du système en parcourant les différents états possibles afin de s'assurer que la logique du code est correcte. Cela permet, d'une certaine manière, de « tester à l'avance » le programme et de vérifier qu'il ne contient pas d'erreurs avant même sa mise en fonctionnement réelle.

Dans la version actuelle du code, ces tests permettent d'identifier les incohérences potentielles. Pour aller plus loin, une amélioration future pourrait consister à afficher un message d'erreur explicite lorsque les tests échouent, et à empêcher le lancement du système. Cela faciliterait les interventions de maintenance et rendrait le programme encore plus robuste et professionnel.

Conclusion et synthèse de la partie Automatisation

Pour conclure cette partie dédiée à l'automatisation, je peux dire que ce projet m'a clairement fait franchir un cap dans ma manière de programmer. Je ne vois plus le code comme une simple suite d'instructions à écrire, mais comme le résultat d'une réflexion structurée en amont.

Ce que je retiens : la rigueur technique

J'ai compris qu'un bon programme se conçoit d'abord sur le papier. Le fait d'avoir validé le diagramme d'états (machine à états) avant d'écrire la moindre ligne de C a rendu la phase de codage beaucoup plus fluide et plus logique.
J'ai également pris conscience de l'importance de la modularité : découper le programme en fonctions bien définies, chacune avec un rôle précis (comme `calculer_etat` ou `calculer_sortie`), rend le code plus lisible, plus compréhensible et plus facile à corriger en cas de problème.

Possibilités d'évolution

Si ce projet devait être poursuivi, deux axes d'amélioration principaux se dégagent :

- **Sécurité logicielle** : actuellement, les tests unitaires signalent une erreur mais n'empêchent pas le programme de continuer son exécution. Une évolution logique serait de bloquer complètement le démarrage du système si un test échoue (principe de *fail-safe*), afin d'éviter toute mise en service dans un état défaillant.

- **Sécurité ferroviaire** : comme évoqué précédemment dans le rapport, je proposerais au client de reconsidérer la demande d'ouverture anticipée de la barrière. Une solution plus sûre consisterait à maintenir la barrière fermée tant que le train n'a pas entièrement quitté la zone et que le système n'est pas revenu à l'état *ATTENTE*. Cela permettrait de mieux prendre en compte des situations particulières, comme des trains atypiques ou des arrêts imprévus.